School of Computer Science and Artificial Intelligence

Lab Assignment # 17.2

Program : B. Tech (CSE)

Specialization :AIML

Course Title : AI Assisted Coding

Course Code : 23CS002PC304

Semester : VI

Academic Session : 2025-2026

Name of Student : D.Sriram

Enrollment No. : 2303A52106

Batch No. : 33

Date :24/03/26

Task 1 – Student Academic Data Cleaning Prompt:

Generate Python code to clean student dataset by handling missing values, removing duplicates, standardizing text, and encoding categorical variables.

Code:

```python
import pandas as pd
import numpy as np

# Creating a sample student DataFrame for demonstration
data = {
    'Student_ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Name': ['Alice Smith', 'Bob Johnson', 'Charlie Brown', 'David Lee', 'Eve Davis', 'Frank White', 'Alice Smith', 'Grace Hall', 'Hei
    'Gender': ['Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female', np.nan, 'Female'],
    'Major': ['Computer Science', 'Mathematics', 'Physics', 'Computer Science', 'Biology', 'Chemistry', 'Computer Science', 'Mathemati
    'GPA': [3.5, 3.8, 3.2, 3.9, np.nan, 3.0, 3.5, 3.7, 3.1, 3.6],
    'Enrollment_Date': ['2020-09-01', '2020-09-01', '2019-09-01', '2021-09-01', '2020-09-01', '2019-09-01', '2020-09-01', '2021-09-01',
    'Email': ['alice@example.com', 'bob@example.com', 'charlie@example.com', 'david@example.com', 'eve@example.com', 'frank@example.cor
}
df = pd.DataFrame(data)

print("Original DataFrame:")
display(df.head())
print(f"Number of rows: {len(df)}")
print(f"Number of columns: {len(df.columns)}")
print("\nDataFrame Info:")
df.info()
```

```
Original DataFrame:
```

	Student_ID	Name	Gender	Major	GPA	Enrollment_Date	Email
0	1	Alice Smith	Female	Computer Science	3.5	2020-09-01	alice@example.com
1	2	Bob Johnson	Male	Mathematics	3.8	2020-09-01	bob@example.com
2	3	Charlie Brown	Male	Physics	3.2	2019-09-01	charlie@example.com
3	4	David Lee	Male	Computer Science	3.9	2021-09-01	david@example.com
4	5	Eve Davis	Female	Biology	NaN	2020-09-01	eve@example.com

```
Number of rows: 10
Number of columns: 7

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Student_ID       10 non-null     int64
 1   Name             10 non-null     object
 2   Gender           9 non-null      object
 3   Major            10 non-null     object
 4   GPA              9 non-null      float64
 5   Enrollment_Date  10 non-null     object
 6   Email            10 non-null     object
dtypes: float64(1), int64(1), object(5)
memory usage: 692.0+ bytes
```

1. Handling Missing Values

First, let's identify the missing values and then decide on a strategy to handle them. Common strategies include:

- **Dropping rows** with missing values (if there are few).
- **Filling missing values** with the mean, median (for numerical data), or mode (for categorical data).
- **Forward-fill** or **Backward-fill** (for time-series or ordered data).

```
print("Missing values before handling:")
display(df.isnull().sum())

# Strategy 1: Fill numerical missing values with the median
# For 'GPA', we will fill NaN with the median GPA.
if 'GPA' in df.columns and df['GPA'].isnull().any():
    median_gpa = df['GPA'].median()
    df['GPA'].fillna(median_gpa, inplace=True)
    print(f"Filled missing 'GPA' values with median: {median_gpa}")

# Strategy 2: Fill categorical missing values with the mode
# For 'Gender', we will fill NaN with the mode (most frequent gender).
if 'Gender' in df.columns and df['Gender'].isnull().any():
    mode_gender = df['Gender'].mode()[0] # .mode() returns a Series, take the first if multiple modes
    df['Gender'].fillna(mode_gender, inplace=True)
    print(f"Filled missing 'Gender' values with mode: {mode_gender}")

print("\nMissing values after handling:")
display(df.isnull().sum())

print("\nDataFrame after handling missing values:")
display(df.head())
```

```
Missing values before handling:
```

	0
Student_ID	0
Name	0
Gender	1
Major	0
GPA	1
Enrollment_Date	0
Email	0

```
dtype: int64
Filled missing 'GPA' values with median: 3.5
Filled missing 'Gender' values with mode: Female

Missing values after handling:
/tmp/ipykernel_679/3642707812.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplac


  df['GPA'].fillna(median_gpa, inplace=True)
/tmp/ipykernel_679/3642707812.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplac


  df['Gender'].fillna(mode_gender, inplace=True)
```

```
df['Gender'].fillna(mode_gender, inplace=True)
```

	0
Student_ID	0
Name	0
Gender	0
Major	0
GPA	0
Enrollment_Date	0
Email	0

```
dtype: int64

DataFrame after handling missing values:
```

	Student_ID	Name	Gender	Major	GPA	Enrollment_Date	Email
0	1	Alice Smith	Female	Computer Science	3.5	2020-09-01	alice@example.com
1	2	Bob Johnson	Male	Mathematics	3.8	2020-09-01	bob@example.com
2	3	Charlie Brown	Male	Physics	3.2	2019-09-01	charlie@example.com
3	4	David Lee	Male	Computer Science	3.9	2021-09-01	david@example.com
4	5	Eve Davis	Female	Biology	3.5	2020-09-01	eve@example.com

2. Removing Duplicates

Duplicate rows can skew analyses. We'll identify and remove them. We might consider certain columns as unique identifiers (e.g., 'Student_ID', 'Email') or check for exact duplicates across all columns.

```
print(f"Number of rows before removing duplicates: {len(df)}")

# Identify duplicates based on all columns or a subset (e.g., 'Student_ID', 'Name', 'Email')
# For this example, we'll consider duplicates based on 'Name' and 'Email' for simplicity,
# as the sample data has 'Alice Smith' duplicated.

df.drop_duplicates(subset=['Name', 'Email'], inplace=True)

print(f"Number of rows after removing duplicates: {len(df)}")
print("\nDataFrame after removing duplicates:")
display(df.head())
```

```
Number of rows before removing duplicates: 10
Number of rows after removing duplicates: 9

DataFrame after removing duplicates:
```

	Student_ID	Name	Gender	Major	GPA	Enrollment_Date	Email
0	1	Alice Smith	Female	Computer Science	3.5	2020-09-01	alice@example.com
1	2	Bob Johnson	Male	Mathematics	3.8	2020-09-01	bob@example.com
2	3	Charlie Brown	Male	Physics	3.2	2019-09-01	charlie@example.com
3	4	David Lee	Male	Computer Science	3.9	2021-09-01	david@example.com
4	5	Eve Davis	Female	Biology	3.5	2020-09-01	eve@example.com

3. Standardizing Text Data

Text standardization typically involves converting text to a consistent case (e.g., lowercase), removing leading/trailing whitespace, and potentially handling special characters or misspellings. For this dataset, we'll standardize 'Name' and 'Major'.

```
print("Text data before standardization:")
display(df[['Name', 'Major']].head())

# Convert to lowercase and strip whitespace for 'Name' and 'Major'
if 'Name' in df.columns:
    df['Name'] = df['Name'].str.lower().str.strip()
    print("Standardized 'Name' column.")
if 'Major' in df.columns:
    df['Major'] = df['Major'].str.lower().str.strip()
    print("Standardized 'Major' column.")

print("\nText data after standardization:")
display(df[['Name', 'Major']].head())
```

```
Text data before standardization:
```

	Name	Major
0	Alice Smith	Computer Science
1	Bob Johnson	Mathematics
2	Charlie Brown	Physics
3	David Lee	Computer Science
4	Eve Davis	Biology

```
Standardized 'Name' column.
Standardized 'Major' column.
```


```
Text data before standardization:
```

	Name	Major
0	Alice Smith	Computer Science
1	Bob Johnson	Mathematics
2	Charlie Brown	Physics
3	David Lee	Computer Science
4	Eve Davis	Biology

```
Standardized 'Name' column.
Standardized 'Major' column.
```


```
Text data after standardization:
```

	Name	Major
0	alice smith	computer science
1	bob johnson	mathematics
2	charlie brown	physics
3	david lee	computer science
4	eve davis	biology

4. Encoding Categorical Variables

Machine learning models often require numerical input. Categorical variables ('Gender', 'Major') need to be converted into numerical representations. Common methods include:

- **One-Hot Encoding**: Creates new binary columns for each category. Suitable for nominal categories (no inherent order).
- **Label Encoding**: Assigns a unique integer to each category. Suitable for ordinal categories (with inherent order) or when the number of categories is very large.

For 'Gender' and 'Major', One-Hot Encoding is generally preferred as there's no inherent order.

```
print("Categorical columns before encoding:")
display(df[['Gender', 'Major']].head())

# One-Hot Encoding for 'Gender' and 'Major'
# Using pandas get_dummies for simplicity
df = pd.get_dummies(df, columns=['Gender', 'Major'], drop_first=True) # drop_first=True avoids multicollinearity

print("\nDataFrame after One-Hot Encoding:")
display(df.head())
print(f"Number of columns after encoding: {len(df.columns)}")
print("\nDataFrame Info after cleaning and encoding:")
df.info()
```

```
Categorical columns before encoding:
```

	Gender	Major
0	Female	computer science
1	Male	mathematics
2	Male	physics
3	Male	computer science
4	Female	biology

```
DataFrame after One-Hot Encoding:
```

	Student_ID	Name	GPA	Enrollment_Date	Email	Gender_Male	Major_chemistry	Major_computer science	Major_mathematics	Major_physics
0	1	alice smith	3.5	2020-09-01	alice@example.com	False	False	True	False	False
1	2	bob johnson	3.8	2020-09-01	bob@example.com	True	False	False	True	False
2	3	charlie brown	3.2	2019-09-01	charlie@example.com	True	False	False	False	True
3	4	david lee	3.9	2021-09-01	david@example.com	True	False	True	False	False
4	5	eve davis	3.5	2020-09-01	eve@example.com	False	False	False	False	False

```
Number of columns after encoding: 10

DataFrame Info after cleaning and encoding:
<class 'pandas.core.frame.DataFrame'>
Index: 9 entries, 0 to 9
Data columns (total 10 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Student_ID              9 non-null      int64
 1   Name                    9 non-null      object
 2   GPA                     9 non-null      float64
 3   Enrollment_Date         9 non-null      object
 4   Email                   9 non-null      object
 5   Gender_Male             9 non-null      bool
 6   Major_chemistry         9 non-null      bool
 7   Major_computer science  9 non-null      bool
 8   Major_mathematics       9 non-null      bool
 9   Major_physics           9 non-null      bool
dtypes: bool(5), float64(1), int64(1), object(3)
memory usage: 477.0+ bytes
```

Task 2 – Financial Transaction Data Preprocessing Prompt:

Generate Python code to preprocess transaction data with datetime conversion, feature extraction, normalization, and outlier handling.

Code:

1. Datetime Conversion and Feature Extraction

Converting date/time columns to datetime objects allows for easy extraction of time-based features (e.g., year, month, day, hour, day of week), which can be crucial for time-series analysis or understanding temporal patterns.

```
]
0s
    print("DataFrame before datetime conversion and feature extraction:")
    display(df[['Transaction_Date']].head())

    # Convert 'Transaction_Date' to datetime objects
    df['Transaction_Date'] = pd.to_datetime(df['Transaction_Date'])

    # Extract time-based features
    df['Year'] = df['Transaction_Date'].dt.year
    df['Month'] = df['Transaction_Date'].dt.month
    df['Day'] = df['Transaction_Date'].dt.day
    df['Hour'] = df['Transaction_Date'].dt.hour
    df['DayOfWeek'] = df['Transaction_Date'].dt.dayofweek # Monday=0, Sunday=6

    print("\nDataFrame after datetime conversion and feature extraction:")
    display(df[['Transaction_Date', 'Year', 'Month', 'Day', 'Hour', 'DayOfWeek']].head())
```

```
...
DataFrame before datetime conversion and feature extraction:
```

	Transaction_Date
0	2023-01-15 10:30:00
1	2023-01-15 11:00:00
2	2023-01-16 14:15:00
3	2023-01-16 15:00:00
4	2023-01-17 09:45:00

```
DataFrame after datetime conversion and feature extraction:
```

	Transaction_Date	Year	Month	Day	Hour	DayOfWeek
0	2023-01-15 10:30:00	2023	1	15	10	6
1	2023-01-15 11:00:00	2023	1	15	11	6
2	2023-01-16 14:15:00	2023	1	16	14	0
3	2023-01-16 15:00:00	2023	1	16	15	0
4	2023-01-17 09:45:00	2023	1	17	9	1

2. Normalization of Numerical Features

Normalization scales numerical features to a standard range (e.g., 0 to 1 or mean 0, standard deviation 1). This is important for many machine learning algorithms that are sensitive to the scale of input features, such as gradient descent-based algorithms or distance-based algorithms like K-Nearest Neighbors.

```
print("Numerical features before normalization:")
display(df[['Amount', 'Item_Count']].head())

# Select numerical features for normalization
numerical_cols = ['Amount', 'Item_Count']

# Initialize StandardScaler
scaler = StandardScaler()

# Apply standardization
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

print("\nNumerical features after normalization (StandardScaler):")
display(df[['Amount', 'Item_Count']].head())
```

```
Numerical features before normalization:
```

	Amount	Item_Count
0	100.50	2
1	25.75	1
2	150.00	3
3	30.20	1
4	500.00	5

```
Numerical features after normalization (StandardScaler):
```

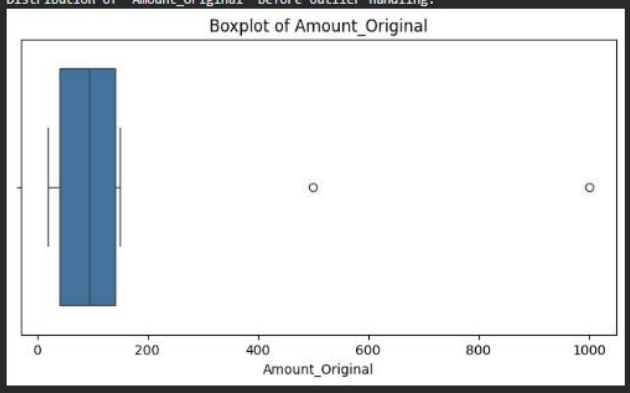
	Amount	Item_Count
0	-0.375797	-0.278543
1	-0.629679	-0.742781
2	-0.207674	0.185695
3	-0.614565	-0.742781
4	0.981072	1.114172

3. Outlier Handling (Example using IQR Method)

Outliers are data points that significantly differ from other observations. They can skew statistical analyses and impact model performance. One common method to identify and handle outliers is the Interquartile Range (IQR) method. Here, we'll demonstrate how to identify outliers and optionally cap them.

```
import matplotlib.pyplot as plt
import seaborn as sns

# For demonstration, let's work with the 'Amount' column before scaling for clarity on original values
# (though in a real scenario, you might do this before or after scaling depending on your strategy)
# Let's re-create a 'raw_amount' column for this demonstration if it was scaled earlier
if 'Amount_Original' not in df.columns:
    df['Amount_Original'] = data['Amount'] # Using the original data amount for outlier detection for clarity

print("Distribution of 'Amount_Original' before outlier handling:")
plt.figure(figsize=(8, 4))
sns.boxplot(x=df['Amount_Original'])
plt.title('Boxplot of Amount_Original')
plt.show()

# Calculate Q1, Q3, and IQR for 'Amount_Original'
Q1 = df['Amount_Original'].quantile(0.25)
Q3 = df['Amount_Original'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers = df[(df['Amount_Original'] < lower_bound) | (df['Amount_Original'] > upper_bound)]

print(f"\nIdentified outliers in 'Amount_Original' (IQR method):")
display(outliers[['Transaction_ID', 'Amount_Original']])

print(f"Lower bound: {lower_bound:.2f}, Upper bound: {upper_bound:.2f}")

# Option to cap outliers (replace with bounds) or remove them
# For this example, we'll cap them. Choose removal if outliers are likely errors or rare events.
df['Amount_Original_Capped'] = df['Amount_Original'].copy()
df.loc[df['Amount_Original_Capped'] < lower_bound, 'Amount_Original_Capped'] = lower_bound
df.loc[df['Amount_Original_Capped'] > upper_bound, 'Amount_Original_Capped'] = upper_bound

print("\nDistribution of 'Amount_Original_Capped' after outlier handling:")
plt.figure(figsize=(8, 4))
sns.boxplot(x=df['Amount_Original_Capped'])
plt.title('Boxplot of Amount_Original_Capped (Outliers Capped)')
plt.show()

display(df[['Amount_Original', 'Amount_Original_Capped']].head())
```

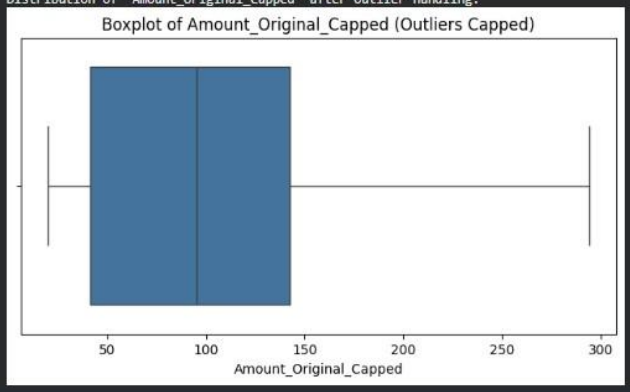
```
Distribution of 'Amount_Original' before outlier handling:
```




```
Identified outliers in 'Amount_Original' (IQR method):
```

	Transaction_ID	Amount_Original
4	5	500.0
9	10	1000.0

```
Lower bound: -110.25, Upper bound: 294.15

Distribution of 'Amount_Original_Capped' after outlier handling:
```



	Amount_Original	Amount_Original_Capped
0	100.50	100.50
1	25.75	25.75
2	150.00	150.00
3	30.20	30.20
4	500.00	294.15

Task 3 – Environmental Sensor Data Preparation

Prompt:

> Generate Python script to clean sensor data, scale features, encode categorical data, and split dataset.

Code:

First, let's create a sample DataFrame representing sensor data. If you already have your data loaded into a DataFrame named `df`, you can skip this cell.

[18] ✓ 0s

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

# Create a sample DataFrame for demonstration of sensor data
data = {
    'Sensor_ID': ['S001', 'S002', 'S003', 'S001', 'S004', 'S005', 'S002', 'S006', 'S007', 'S008'],
    'Timestamp': pd.to_datetime([
        '2023-10-26 08:00:00', '2023-10-26 08:05:00', '2023-10-26 08:10:00',
        '2023-10-26 08:00:00', '2023-10-26 08:15:00', '2023-10-26 08:20:00',
        '2023-10-26 08:05:00', '2023-10-26 08:25:00', '2023-10-26 08:30:00',
        '2023-10-26 08:35:00'
    ]),
    'Temperature': [25.1, 26.5, 24.9, 25.0, np.nan, 27.0, 26.3, 25.5, 24.0, 28.1],
    'Humidity': [60.2, 62.1, 59.8, 60.0, 65.0, 63.5, 62.0, np.nan, 58.5, 64.0],
    'Pressure': [1012.5, 1013.0, 1011.8, 1012.4, 1010.5, 1014.0, 1012.9, 1011.0, 1010.0, 1015.0],
    'Status': ['Green', 'Yellow', 'Green', 'Green', 'Red', 'Yellow', 'Green', 'Red', 'Green', 'Yellow'],
    'Location': ['Lab1', 'Lab2', 'Lab1', 'Lab1', 'Lab3', 'Lab2', 'Lab2', 'Lab1', 'Lab3', 'Lab1']
}
df = pd.DataFrame(data)

print("Original Sensor Data DataFrame:")
display(df.head())
print(f"Number of rows: {len(df)}")
print(f"Number of columns: {len(df.columns)}")
print("\nDataFrame Info:")
df.info()
```

```
Original Sensor Data DataFrame:
```

	Sensor_ID	Timestamp	Temperature	Humidity	Pressure	Status	Location
0	S001	2023-10-26 08:00:00	25.1	60.2	1012.5	Green	Lab1
1	S002	2023-10-26 08:05:00	26.5	62.1	1013.0	Yellow	Lab2
2	S003	2023-10-26 08:10:00	24.9	59.8	1011.8	Green	Lab1
3	S001	2023-10-26 08:00:00	25.0	60.0	1012.4	Green	Lab1
4	S004	2023-10-26 08:15:00	NaN	65.0	1010.5	Red	Lab3

```
Number of rows: 10
Number of columns: 7

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 7 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Sensor_ID    10 non-null     object
 1   Timestamp    10 non-null     datetime64[ns]
 2   Temperature  9 non-null      float64
 3   Humidity     9 non-null      float64
 4   Pressure     10 non-null     float64
 5   Status       10 non-null     object
 6   Location     10 non-null     object
dtypes: datetime64[ns](1), float64(3), object(3)
memory usage: 692.0+ bytes
```

1. Cleaning Sensor Data

This step involves handling missing values and removing duplicate entries. For sensor data, missing values often need careful consideration (e.g., interpolation, mean/median/mode imputation). Duplicates might indicate faulty sensors or logging errors.

```python
print("Missing values before cleaning:")
display(df.isnull().sum())

# Handle missing numerical values with the median
for col in ['Temperature', 'Humidity', 'Pressure']:
    if col in df.columns and df[col].isnull().any():
        median_val = df[col].median()
        df[col].fillna(median_val, inplace=True)
        print(f"Filled missing '{col}' values with median: {median_val}")

print("\nMissing values after imputation:")
display(df.isnull().sum())

print(f"\nNumber of rows before removing duplicates: {len(df)}")
# Remove duplicates based on 'Sensor_ID' and 'Timestamp'
df.drop_duplicates(subset=['Sensor_ID', 'Timestamp'], inplace=True)
print(f"Number of rows after removing duplicates: {len(df)}")

print("\nDataFrame after cleaning:")
display(df.head())
```

```
Missing values before cleaning:
```

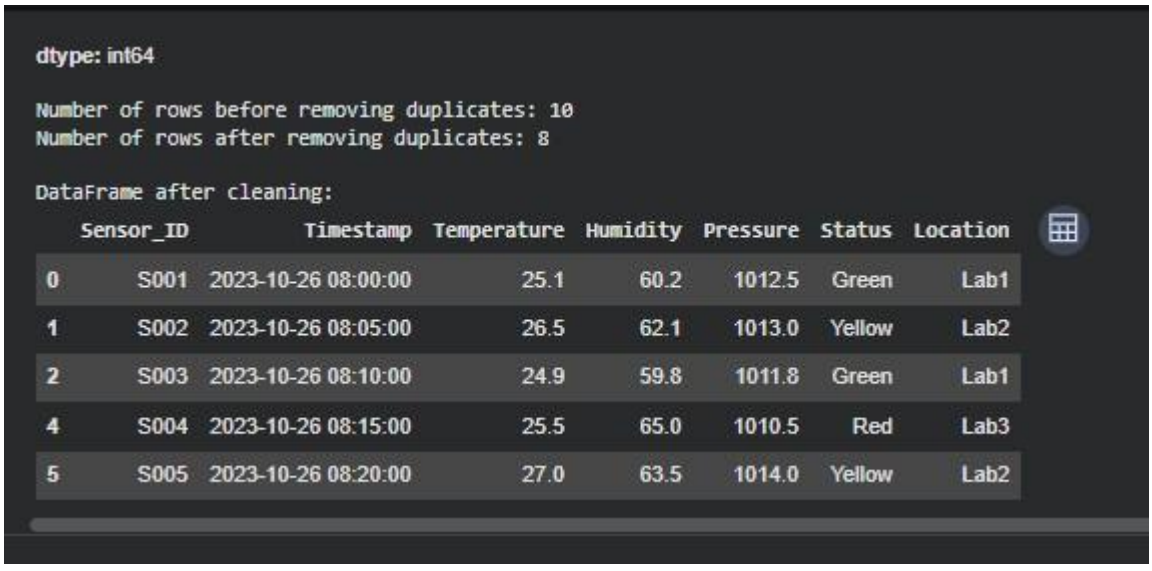
	0
Sensor_ID	0
Timestamp	0
Temperature	1
Humidity	1
Pressure	0
Status	0
Location	0

```
dtype: int64
Filled missing 'Temperature' values with median: 25.5
Filled missing 'Humidity' values with median: 62.0

Missing values after imputation:
/tmp/ipykernel_679/1815750461.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operatic


  df[col].fillna(median_val, inplace=True)
```

	0
Sensor_ID	0
Timestamp	0
Temperature	0
Humidity	0
Pressure	0
Status	0
Location	0

```
dtype: int64

Number of rows before removing duplicates: 10
Number of rows after removing duplicates: 8

DataFrame after cleaning:
```

	Sensor_ID	Timestamp	Temperature	Humidity	Pressure	Status	Location
0	S001	2023-10-26 08:00:00	25.1	60.2	1012.5	Green	Lab1
1	S002	2023-10-26 08:05:00	26.5	62.1	1013.0	Yellow	Lab2



dtype: int64

Number of rows before removing duplicates: 10
Number of rows after removing duplicates: 8

DataFrame after cleaning:

	Sensor_ID	Timestamp	Temperature	Humidity	Pressure	Status	Location
0	S001	2023-10-26 08:00:00	25.1	60.2	1012.5	Green	Lab1
1	S002	2023-10-26 08:05:00	26.5	62.1	1013.0	Yellow	Lab2
2	S003	2023-10-26 08:10:00	24.9	59.8	1011.8	Green	Lab1
4	S004	2023-10-26 08:15:00	25.5	65.0	1010.5	Red	Lab3
5	S005	2023-10-26 08:20:00	27.0	63.5	1014.0	Yellow	Lab2

3. Encoding Categorical Data

Machine learning models require numerical input. Categorical features like 'Status' and 'Location' need to be converted into a numerical format. One-hot encoding creates binary columns for each category, which is suitable when there is no inherent order among categories.

```
categorical_features = ['Status', 'Location']

# Initialize OneHotEncoder
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# Fit and transform the categorical features
encoded_features = encoder.fit_transform(df[categorical_features])

# Create a DataFrame from the encoded features
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(categorical_features))

# Reset index of original DataFrame to ensure proper concatenation
df.reset_index(drop=True, inplace=True)

# Concatenate the encoded features with the original DataFrame (dropping original categorical columns)
df = pd.concat([df.drop(columns=categorical_features), encoded_df], axis=1)

print("DataFrame after One-Hot Encoding (first 5 rows):")
display(df.head())
print(f"Number of columns after encoding: {len(df.columns)}")
print("\nDataFrame Info after cleaning, scaling, and encoding:")
df.info()
```

```
... DataFrame after One-Hot Encoding (first 5 rows):
```

```
DataFrame after One-Hot Encoding (first 5 rows):
```

	Sensor_ID	Timestamp	Temperature	Humidity	Pressure	Status_Green	Status_Red	Status_Yellow	Location_Lab1	Location_Lab2	Location_Lab3
0	S001	2023-10-26 08:00:00	-0.593570	-0.801205	0.169191	1.0	0.0	0.0	1.0	0.0	0.0
1	S002	2023-10-26 08:05:00	0.552634	0.100892	0.476810	0.0	0.0	1.0	0.0	1.0	0.0
2	S003	2023-10-26 08:10:00	-0.757313	-0.991120	-0.261477	1.0	0.0	0.0	1.0	0.0	0.0
3	S004	2023-10-26 08:15:00	-0.266083	1.477777	-1.061287	0.0	1.0	0.0	0.0	0.0	1.0
4	S005	2023-10-26 08:20:00	0.961992	0.765595	1.092049	0.0	0.0	1.0	0.0	1.0	0.0

```
Number of columns after encoding: 11

DataFrame Info after cleaning, scaling, and encoding:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 11 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Sensor_ID      8 non-null      object
 1   Timestamp      8 non-null      datetime64[ns]
 2   Temperature    8 non-null      float64
 3   Humidity       8 non-null      float64
 4   Pressure       8 non-null      float64
 5   Status_Green   8 non-null      float64
 6   Status_Red     8 non-null      float64
 7   Status_Yellow  8 non-null      float64
 8   Location_Lab1  8 non-null      float64
 9   Location_Lab2  8 non-null      float64
 10  Location_Lab3  8 non-null      float64
dtypes: datetime64[ns](1), float64(9), object(1)
memory usage: 836.0+ bytes
```

4. Splitting the Dataset

Finally, the dataset needs to be split into training and testing sets. This is a critical step for evaluating the performance of machine learning models on unseen data. A common split ratio is 80% for training and 20% for testing.

```
# For demonstration, let's assume 'Temperature' is our target variable for a prediction task.
# In a real scenario, you would define your features (X) and target (y) based on your specific problem.

# Drop non-feature columns that are not relevant for splitting if they exist
X = df.drop(columns=['Sensor_ID', 'Timestamp'], errors='ignore') # Assuming Sensor_ID and Timestamp are identifiers, not features for splitting

# If 'Temperature' is a feature for a different prediction, then X would include it.
# For this example, let's assume we want to predict a hypothetical 'Fault_Indicator' or similar if available.
# As a generic example, we'll split without a specific 'y' to demonstrate general dataset splitting.
# If you have a target variable, replace this X and y definition.

# For supervised learning, you'd typically have a target variable 'y'.
# Let's assume 'Temperature' (after scaling) is the target for a regression task for demonstration purposes.
# If you don't have a specific target, you can skip separating X and y.

if 'Temperature' in X.columns:
    y = X['Temperature']
    X = X.drop(columns=['Temperature'])
else:
    # If 'Temperature' was already removed or not intended as target, just use all remaining columns as X
    y = pd.Series(np.random.rand(len(X)), index=X.index) # Placeholder target if none explicit

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f"Shape of training features (X_train): {X_train.shape}")
print(f"Shape of testing features (X_test): {X_test.shape}")
print(f"Shape of training target (y_train): {y_train.shape}")
print(f"Shape of testing target (y_test): {y_test.shape}")

print("\nFirst 5 rows of X_train:")
display(X_train.head())
```

```
Shape of training features (X_train): (6, 8)
Shape of testing features (X_test): (2, 8)
Shape of training target (y_train): (6,)
Shape of testing target (y_test): (2,)

First 5 rows of X_train:
```

	Humidity	Pressure	Status_Green	Status_Red	Status_Yellow	Location_Lab1	Location_Lab2	Location_Lab3
0	-0.801205	0.169191	1.0	0.0	0.0	1.0	0.0	0.0
7	1.002989	1.707288	0.0	0.0	1.0	1.0	0.0	0.0
2	-0.991120	-0.261477	1.0	0.0	0.0	1.0	0.0	0.0
4	0.765595	1.092049	0.0	0.0	1.0	0.0	1.0	0.0
3	1.477777	-1.061287	0.0	1.0	0.0	0.0	0.0	1.0

Task 4 – Smart City Traffic Data Cleaning Prompt:

Generate Python script to clean traffic data including normalization, missing values, duplicates, and summary comparison.

Code:

First, let's create a sample DataFrame representing traffic data. If you already have your data loaded into a DataFrame named `df`, you can skip this cell.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# Create a sample DataFrame for demonstration of traffic data
data = {
    'Timestamp': pd.to_datetime([
        '2023-09-01 07:00:00', '2023-09-01 07:05:00', '2023-09-01 07:10:00',
        '2023-09-01 07:00:00', '2023-09-01 07:15:00', '2023-09-01 07:20:00',
        '2023-09-01 07:25:00', '2023-09-01 07:20:00', '2023-09-01 07:30:00',
        '2023-09-01 07:35:00'
    ]),
    'Location_ID': ['L001', 'L002', 'L001', 'L001', 'L003', 'L002', 'L001', 'L002', 'L003', 'L001'],
    'Traffic_Volume': [150, 200, 160, 150, np.nan, 210, 170, 210, 190, 220],
    'Speed_kmh': [60, 45, 55, 60, 70, np.nan, 50, 48, 65, 40],
    'Weather_Condition': ['Clear', 'Rainy', 'Clear', 'Clear', 'Foggy', 'Clear', 'Rainy', 'Clear', 'Clear', np.nan]
}
df_traffic = pd.DataFrame(data)

print("Original Traffic Data DataFrame:")
display(df_traffic.head())
print(f"Number of rows: {len(df_traffic)}")
print(f"Number of columns: {len(df_traffic.columns)}")
print("\nDataFrame Info:")
df_traffic.info()

# Keep a copy of the original DataFrame for comparison later
df_traffic_original = df_traffic.copy()
```

```
Original Traffic Data DataFrame:
```

	Timestamp	Location_ID	Traffic_Volume	Speed_kmh	Weather_Condition
0	2023-09-01 07:00:00	L001	150.0	60.0	Clear
1	2023-09-01 07:05:00	L002	200.0	45.0	Rainy
2	2023-09-01 07:10:00	L001	160.0	55.0	Clear
3	2023-09-01 07:00:00	L001	150.0	60.0	Clear
4	2023-09-01 07:15:00	L003	NaN	70.0	Foggy

```
Number of rows: 10
Number of columns: 5

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   Timestamp         10 non-null     datetime64[ns]
 1   Location_ID       10 non-null     object
 2   Traffic_Volume    9 non-null      float64
 3   Speed_kmh         9 non-null      float64
 4   Weather_Condition 9 non-null      object
dtypes: datetime64[ns](1), float64(2), object(2)
memory usage: 532.0+ bytes
```

1. Handling Missing Values

Missing values are common in real-world datasets. For numerical features like `Traffic_Volume` and `Speed_kmh`, we'll fill them with the median. For categorical features like `Weather_Condition`, we'll use the mode.

```
print("Missing values before handling:")
display(df_traffic.isnull().sum())

# Fill numerical missing values with the median
for col in ['Traffic_Volume', 'Speed_kmh']:
    if col in df_traffic.columns and df_traffic[col].isnull().any():
        median_val = df_traffic[col].median()
        df_traffic[col].fillna(median_val, inplace=True)
        print(f"Filled missing '{col}' values with median: {median_val}")

# Fill categorical missing values with the mode
if 'Weather_Condition' in df_traffic.columns and df_traffic['Weather_Condition'].isnull().any():
    mode_val = df_traffic['Weather_Condition'].mode()[0] # .mode() returns a Series, take the first
    df_traffic['Weather_Condition'].fillna(mode_val, inplace=True)
    print(f"Filled missing 'Weather_Condition' values with mode: {mode_val}")

print("\nMissing values after handling:")
display(df_traffic.isnull().sum())

print("\nDataFrame after handling missing values (head):")
display(df_traffic.head())
```

```
Missing values before handling:
```

	0
Timestamp	0
Location_ID	0
Traffic_Volume	1
Speed_kmh	1
Weather_Condition	1

```
dtype: int64
Filled missing 'Traffic_Volume' values with median: 190.0
Filled missing 'Speed_kmh' values with median: 55.0
Filled missing 'Weather_Condition' values with mode: Clear

Missing values after handling:
/tmp/ipykernel_679/1698518030.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Serie
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on w

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'


  df_traffic[col].fillna(median_val, inplace=True)
/tmp/ipykernel_679/1698518030.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Seri
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on w

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)'


  df_traffic['Weather_Condition'].fillna(mode_val, inplace=True)
```

	0
Timestamp	0
Location_ID	0
Traffic_Volume	0
Speed_kmh	0
Weather_Condition	0

dtype: int64

```
DataFrame after handling missing values (head):
```

	Timestamp	Location_ID	Traffic_Volume	Speed_kmh	Weather_Condition
0	2023-09-01 07:00:00	L001	150.0	60.0	Clear
1	2023-09-01 07:05:00	L002	200.0	45.0	Rainy
2	2023-09-01 07:10:00	L001	160.0	55.0	Clear
3	2023-09-01 07:00:00	L001	150.0	60.0	Clear
4	2023-09-01 07:15:00	L003	190.0	70.0	Foggy

2. Removing Duplicates

Duplicate entries can bias analyses. We'll identify and remove duplicates based on a combination of `Timestamp` and `Location_ID`, as these are expected to be unique for each observation.

```
print(f"Number of rows before removing duplicates: {len(df_traffic)}")

# Remove duplicates based on 'Timestamp' and 'Location_ID'
df_traffic.drop_duplicates(subset=['Timestamp', 'Location_ID'], inplace=True)

print(f"Number of rows after removing duplicates: {len(df_traffic)}")
print("\nDataFrame after removing duplicates (head):")
display(df_traffic.head())
```

```
Number of rows before removing duplicates: 10
Number of rows after removing duplicates: 8

DataFrame after removing duplicates (head):
```

	Timestamp	Location_ID	Traffic_Volume	Speed_kmh	Weather_Condition
0	2023-09-01 07:00:00	L001	150.0	60.0	Clear
1	2023-09-01 07:05:00	L002	200.0	45.0	Rainy
2	2023-09-01 07:10:00	L001	160.0	55.0	Clear
4	2023-09-01 07:15:00	L003	190.0	70.0	Foggy
5	2023-09-01 07:20:00	L002	210.0	55.0	Clear

3. Normalization of Numerical Features

Normalization (using StandardScaler) is important for numerical features to ensure they contribute equally to models and prevent features with larger ranges from dominating. We'll apply this to `Traffic_Volume` and `Speed_kmh`.

```
print("Numerical features before normalization (mean and std):")
display(df_traffic[['Traffic_Volume', 'Speed_kmh']].agg(['mean', 'std']))

numerical_cols = ['Traffic_Volume', 'Speed_kmh']

# Initialize StandardScaler
scaler = StandardScaler()

# Apply standardization
df_traffic[numerical_cols] = scaler.fit_transform(df_traffic[numerical_cols])

print("\nNumerical features after normalization (StandardScaler - mean and std):")
display(df_traffic[['Traffic_Volume', 'Speed_kmh']].agg(['mean', 'std']))

print("\nDataFrame after normalization (head):")
display(df_traffic.head())
```

```
Numerical features before normalization (mean and std):
```

	Traffic_Volume	Speed_kmh
mean	186.25000	55.0
std	24.45842	10.0

```
Numerical features after normalization (StandardScaler - mean and std):
```

	Traffic_Volume	Speed_kmh
mean	2.775558e-17	0.000000
std	1.069045e+00	1.069045

```
DataFrame after normalization (head):
```

	Timestamp	Location_ID	Traffic_Volume	Speed_kmh	Weather_Condition
0	2023-09-01 07:00:00	L001	-1.584439	0.534522	Clear
1	2023-09-01 07:05:00	L002	0.600994	-1.069045	Rainy
2	2023-09-01 07:10:00	L001	-1.147353	0.000000	Clear
4	2023-09-01 07:15:00	L003	0.163908	1.603567	Foggy
5	2023-09-01 07:20:00	L002	1.038081	0.000000	Clear

4. Summary Comparison (Before and After Preprocessing)

Let's compare the characteristics of the original and processed DataFrames to see the impact of our cleaning and normalization steps.

```
print("--- Original DataFrame Summary ---")
df_traffic_original.info()
print("\nMissing values in original:")
display(df_traffic_original.isnull().sum())
print("\nDescription of original numerical features:")
display(df_traffic_original[['Traffic_Volume', 'Speed_kmh']].describe())

print("\n\n--- Processed DataFrame Summary ---")
df_traffic.info()
print("\nMissing values in processed:")
display(df_traffic.isnull().sum())
print("\nDescription of processed numerical features:")
display(df_traffic[['Traffic_Volume', 'Speed_kmh']].describe())
```

```
--- Original DataFrame Summary ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Timestamp          10 non-null     datetime64[ns]
 1   Location_ID        10 non-null     object
 2   Traffic_Volume     9 non-null      float64
 3   Speed_kmh          9 non-null      float64
 4   Weather_Condition  9 non-null      object
dtypes: datetime64[ns](1), float64(2), object(2)
memory usage: 532.0+ bytes

Missing values in original:
```

	0
Timestamp	0
Location_ID	0
Traffic_Volume	1
Speed_kmh	1
Weather_Condition	1

dtype: int64

```
Description of original numerical features:
```

	Traffic_Volume	Speed_kmh
count	9.000000	9.000000
mean	184.444444	54.777778

Description of original numerical features:

	Traffic_Volume	Speed_kmh
count	9.000000	9.000000
mean	184.444444	54.777778
std	27.436796	9.833333
min	150.000000	40.000000
25%	160.000000	48.000000
50%	190.000000	55.000000
75%	210.000000	60.000000
max	220.000000	70.000000

```
--- Processed DataFrame Summary ---
<class 'pandas.core.frame.DataFrame'>
Index: 8 entries, 0 to 9
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   Timestamp          8 non-null      datetime64[ns]
 1   Location_ID        8 non-null      object
 2   Traffic_Volume     8 non-null      float64
 3   Speed_kmh          8 non-null      float64
 4   Weather_Condition  8 non-null      object
dtypes: datetime64[ns](1), float64(2), object(2)
memory usage: 384.0+ bytes
```

Missing values in processed:

	0
Timestamp	0
Location_ID	0
Traffic_Volume	0
Speed_kmh	0
Weather_Condition	0

dtype: int64

Description of processed numerical features:

	Traffic_Volume	Speed_kmh
count	8.000000e+00	8.000000
mean	2.775558e-17	0.000000
std	1.069045e+00	1.069045
min	-1.584439e+00	-1.603567
25%	-8.195375e-01	-0.668153
50%	1.639075e-01	0.000000
75%	7.102659e-01	0.668153
max	1.475168e+00	1.603567

Task 5 – Social Media Text Cleaning Prompt:

Generate Python code to clean text data, remove stopwords, tokenize, and extract features for sentiment analysis.

Code:

```python
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer

# Download necessary NLTK data (run once)
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')
try:
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')

# Create a sample DataFrame for demonstration of text data
data = {
    'Review_ID': range(1, 11),
    'Text_Review': [
        'This product is amazing! I love it so much. Best purchase ever.',
        'It was okay, but not great. The quality could be better.',
        'Terrible experience. Absolutely regret buying this.',
        'Good value for money. Highly recommend to everyone.',
        'Mediocre. Expected more given the price. 2/5 stars.',
        'Fantastic! Exceeded all my expectations. Will buy again.',
        'Very bad. Broke after a week. Do not recommend.',
        'Decent product, works as advertised. Nothing special.',
        'Loved it! Super happy with the performance and design.',
        'Disappointing. Had high hopes but it failed to deliver.'
    ],
    'Sentiment': ['Positive', 'Neutral', 'Negative', 'Positive', 'Negative', 'Positive', 'Negative', 'Neutral', 'Positive', 'Negative']
}
df_text = pd.DataFrame(data)

print("Original Text Data DataFrame:")
display(df_text.head())
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
Original Text Data DataFrame:
```

	Review_ID	Text_Review	Sentiment
0	1	This product is amazing! I love it so much. Be...	Positive
1	2	It was okay, but not great. The quality could ...	Neutral
2	3	Terrible experience. Absolutely regret buying ...	Negative
3	4	Good value for money. Highly recommend to ever...	Positive
4	5	Mediocre. Expected more given the price. 2/5 s...	Negative

```
Number of rows: 10
Number of columns: 3

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Review_ID    10 non-null     int64
 1   Text_Review  10 non-null     object
 2   Sentiment    10 non-null     object
dtypes: int64(1), object(2)
memory usage: 372.0+ bytes
```

First, let's create a sample DataFrame with text data to demonstrate the preprocessing steps for sentiment analysis. If you already have your data loaded into a DataFrame named `df_text`, you can skip this cell.

```python
import pandas as pd
import re
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer

# Download necessary NLTK data (run once)
try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')
try:
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')

# Explicitly download 'punkt_tab' as suggested by the error message
# This is done unconditionally to ensure it's present if NLTK expects it separately.
nltk.download('punkt_tab')

# Create a sample DataFrame for demonstration of text data
data = {
    'Review_ID': range(1, 11),
    'Text_Review': [
        'This product is amazing! I love it so much. Best purchase ever.',
        'It was okay, but not great. The quality could be better.',
        'Terrible experience. Absolutely regret buying this.',
        'Good value for money. Highly recommend to everyone.',
        'Mediocre. Expected more given the price. 2/5 stars.',
        'Fantastic! Exceeded all my expectations. Will buy again.',
        'Very bad. Broke after a week. Do not recommend.',
        'Decent product, works as advertised. Nothing special.',
        'Loved it! Super happy with the performance and design.',
        'Disappointing. Had high hopes but it failed to deliver.'
    ]
}
```

```
    ],
    'Sentiment': ['Positive', 'Neutral', 'Negative', 'Positive', 'Negative', 'Positive', 'Negative', 'Neutral', 'Positive', 'Nega
}
df_text = pd.DataFrame(data)

print("Original Text Data DataFrame:")
display(df_text.head())
print(f"Number of rows: {len(df_text)}")
print(f"Number of columns: {len(df_text.columns)}")
print("\nDataFrame Info:")
df_text.info()
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
Original Text Data DataFrame:
```

	Review_ID	Text_Review	Sentiment
0	1	This product is amazing! I love it so much. Be...	Positive
1	2	It was okay, but not great. The quality could ...	Neutral
2	3	Terrible experience. Absolutely regret buying ...	Negative
3	4	Good value for money. Highly recommend to ever...	Positive
4	5	Mediocre. Expected more given the price. 2/5 s...	Negative

```
Number of rows: 10
Number of columns: 3

DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 3 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Review_ID    10 non-null     int64
 1   Text_Review  10 non-null     object
 2   Sentiment    10 non-null     object
dtypes: int64(1), object(2)
memory usage: 372.0+ bytes
```

1. Text Cleaning

This step involves preprocessing the raw text data by converting it to lowercase, removing special characters, numbers, and extra whitespace. This standardizes the text and helps reduce noise for better analysis.

```
def clean_text(text):
    text = text.lower() # Convert to lowercase
    text = re.sub(r'[^a-z\s]', '', text) # Remove special characters and numbers (keep only letters and spaces)
    text = re.sub(r'\s+', ' ', text).strip() # Remove extra spaces
    return text

print("Original text reviews:")
display(df_text['Text_Review'].head())

df_text['Clean_Review'] = df_text['Text_Review'].apply(clean_text)

print("\nCleaned text reviews:")
display(df_text['Clean_Review'].head())
```

```
Original text reviews:
```

	Text_Review
0	This product is amazing! I love it so much. Be...
1	It was okay, but not great. The quality could ...
2	Terrible experience. Absolutely regret buying ...
3	Good value for money. Highly recommend to ever...
4	Mediocre. Expected more given the price. 2/5 s...

dtype: object

```
Cleaned text reviews:
```

	Clean_Review
0	this product is amazing i love it so much best...
1	it was okay but not great the quality could be...
2	terrible experience absolutely regret buying this
3	good value for money highly recommend to everyone
4	mediocre expected more given the price stars

dtype: object

2. Stopword Removal and Tokenization

Stopword Removal: Stopwords are common words (like 'the', 'is', 'a') that often do not carry significant meaning for sentiment analysis. Removing them can reduce the dimensionality of the data and improve model performance.

Tokenization: This is the process of breaking down text into individual words or units, called tokens. This is a fundamental step before most text processing tasks.

[27] 0s

```python
stop_words = set(stopwords.words('english'))

def remove_stopwords_and_tokenize(text):
    tokens = word_tokenize(text) # Tokenize the text
    filtered_tokens = [word for word in tokens if word not in stop_words] # Remove stopwords
    return filtered_tokens

print("Cleaned reviews before tokenization and stopword removal:")
display(df_text['Clean_Review'].head())

df_text['Tokens'] = df_text['Clean_Review'].apply(remove_stopwords_and_tokenize)

print("\nReviews after tokenization and stopword removal:")
display(df_text['Tokens'].head())
```

```
Cleaned reviews before tokenization and stopword removal:
```

Clean_Review

```
Cleaned reviews before tokenization and stopword removal:
```

	Clean_Review
0	this product is amazing i love it so much best...
1	it was okay but not great the quality could be...
2	terrible experience absolutely regret buying this
3	good value for money highly recommend to everyone
4	mediocre expected more given the price stars

dtype: object

```
Reviews after tokenization and stopword removal:
```

	Tokens
0	[product, amazing, love, much, best, purchase,...
1	[okay, great, quality, could, better]
2	[terrible, experience, absolutely, regret, buy...
3	[good, value, money, highly, recommend, everyone]
4	[mediocre, expected, given, price, stars]

dtype: object

3. Feature Extraction (TF-IDF)

To use text data in machine learning models, it needs to be converted into a numerical format. TF-IDF (Term Frequency-Inverse Document Frequency) is a common technique that reflects how important a word is to a document in a corpus. It assigns a weight to each word based on its frequency within a document and its rarity across all documents.

[28] ✓ 0s

```python
# Join tokens back into a single string for TF-IDF Vectorizer
df_text['Processed_Text'] = df_text['Tokens'].apply(lambda x: ' '.join(x))

# Initialize TF-IDF Vectorizer
tfidf_vectorizer = TfidfVectorizer(max_features=100) # Limit to top 100 features for demonstration

# Fit and transform the processed text data
tfidf_features = tfidf_vectorizer.fit_transform(df_text['Processed_Text'])

# Convert TF-IDF features to a DataFrame for easier inspection
tfidf_df = pd.DataFrame(tfidf_features.toarray(), columns=tfidf_vectorizer.get_feature_names_out())

print("TF-IDF Features (first 5 rows and a few columns):")
display(tfidf_df.head())
print(f"Shape of TF-IDF features: {tfidf_df.shape}")
```

```
TF-IDF Features (first 5 rows and a few columns):
```

	absolutely	advertised	amazing	bad	best	better	broke	buy	buying	could	...	quality	recommend	regret	speci
0	0.000000	0.0	0.385682	0.0	0.385682	0.000000	0.0	0.0	0.000000	0.000000	...	0.000000	0.000000	0.000000	0
1	0.000000	0.0	0.000000	0.0	0.000000	0.447214	0.0	0.0	0.000000	0.447214	...	0.447214	0.000000	0.000000	0
2	0.447214	0.0	0.000000	0.0	0.000000	0.000000	0.0	0.0	0.447214	0.000000	...	0.000000	0.000000	0.447214	0
3	0.000000	0.0	0.000000	0.0	0.000000	0.000000	0.0	0.0	0.000000	0.000000	...	0.000000	0.355359	0.000000	0
4	0.000000	0.0	0.000000	0.0	0.000000	0.000000	0.0	0.0	0.000000	0.000000	...	0.000000	0.000000	0.000000	0

5 rows × 50 columns

```
Shape of TF-IDF features: (10, 50)
```